

 > Lessons > Build your own decentralized exchange like Uniswap v1



LearnWeb3

11 min read · 3 years ago

+10,000 XP



429



51



18



Build your own decentralized exchange like Uniswap v1

Previously, we learnt about how decentralized exchanges like Uniswap v1 work. If you haven't gone through that lesson yet - go read it first and come back as it explains the mathematics behind an AMM like Uniswap v1. Now, we're going to use that knowledge to build our own decentralized exchange from scratch that allows swapping ETH <> TOKEN.

Requirements

- Build an exchange that allows swapping ETH <> TOKEN
- DEX must charge a 1% fee on swaps
- When user adds liquidity, they must be given an LP Token that represents their share of the pool



- LP must be able to burn their LP tokens to receive back ETH and TOKEN

Foundry Setup

We will be building out the smart contracts necessary for this project. We will use Foundry for the smart contracts.

Start off by creating a new folder on your computer - I named mine `dex-app`.

Open up a Terminal pointing to the `dex-app` folder and run the following commands:

```
forge init foundry-app
```



We also need to install OpenZeppelin's contracts as we will use it to build out the ERC-20 Token. Run the following commands:

```
cd foundry-app  
forge install OpenZeppelin/openzeppelin-contracts
```



To configure remappings in your project and make sure they're picked up when compiling your code, run the following command:

```
forge remappings > remappings.txt
```



Environment Variables

Start off by going to the `.env` file inside the `foundry-app` folder and add the following placeholder lines:

```
PRIVATE_KEY="..."
RPC_URL="..."
ETHERSCAN_API_KEY="..."
```



For the `PRIVATE_KEY` variable, export it from MetaMask. Again, make sure to use an account that only has testnet funds in it, no mainnet funds, to not risk accidentally leaking a private key with real money in it. Replace the value of `PRIVATE_KEY` in the `.env` file with the key you export.

For the `RPC_URL`, create an account at [QuickNode](#) if you don't have one already. After creating an account, click on `Create an endpoint`, select `Ethereum`, and then select the `Sepolia` testnet. Continue with the process until finished, and then copy the `HTTP Provider` link from the dashboard there.

Replace the value of the `RPC_URL` variable in the `.env` file with the `HTTP Provider` link you copied.

NOTE: If you have previously set up a Sepolia endpoint on QuickNode during an earlier lesson, you can continue using that. You do not need to create a new one.

Lastly, we need an Etherscan API Key so our contract can be verified on Etherscan. You can grab an Etherscan API Key by creating an account on <https://etherscan.io> if you don't have one already. Once you have it, replace the value of `ETHERSCAN_API_KEY` in your `.env` file.



Making an ERC-20 Token

Now, let's start off by creating a really simple ERC-20 Token that will be used to create the trading pool on our exchange. We will just use a dummy token.

Create a new file named Token.sol under foundry-app/src and write the following code there:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.25;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract Token is ERC20 {
    // Initialize contract with 1 million tokens minted to the creator of the contract
    constructor() ERC20("Token", "TKN") {
        _mint(msg.sender, 1000000 * 10 ** decimals());
    }
}
```

This contract is a very basic ERC-20 that just mints a million tokens to the deployer address.

Making the Exchange Contract

Now for the fun part. Create a file named "Exchange.sol" under `foundry-app/src` and write the following starter code there:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.25;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
```



```
contract Exchange is ERC20 {  
    // Future code goes here  
}
```

Now, let's go step by step in adding more functionality:

First, let's create a `constructor` for the contract. This constructor takes the contract address of the token and saves it. The exchange will then further behave as an exchange for ETH <> Token.

```
address public tokenAddress;  
  
// Exchange is inheriting ERC20, because our exchange itself is an ERC-20 contract  
// as it is responsible for minting and issuing LP Tokens  
constructor(address token) ERC20("ETH TOKEN LP Token", "lpETHTOKEN") {  
    require(token != address(0), "Token address passed is a null address");  
    tokenAddress = token;  
}
```

Now, let's create a simply `view` function that returns the balance of Token for the exchange contract itself i.e. how many `Token` s are in the exchange contract.

```
// getReserve returns the balance of `token` held by `this` contract  
function getReserve() public view returns (uint256) {  
    return ERC20(tokenAddress).balanceOf(address(this));  
}
```



Now, let's create the function to add liquidity to the exchange. This code is explained in the comments. If you are confused about how we are calculating certain things - refer back to the lesson which introduced how decentralized exchanges work and went over the math equations.

```
// addLiquidity allows users to add liquidity to the exchange
function addLiquidity(
    uint256 amountOfToken
) public payable returns (uint256) {
    uint256 lpTokensToMint;
    uint256 ethReserveBalance = address(this).balance;
    uint256 tokenReserveBalance = getReserve();

    ERC20 token = ERC20(tokenAddress);

    // If the reserve is empty, take any user supplied value for initial liquidity
    if (tokenReserveBalance == 0) {
        // Transfer the token from the user to the exchange
        token.transferFrom(msg.sender, address(this), amountOfToken);

        // lpTokensToMint = ethReserveBalance = msg.value
        lpTokensToMint = ethReserveBalance;

        // Mint LP tokens to the user
        _mint(msg.sender, lpTokensToMint);

        return lpTokensToMint;
    }

    // If the reserve is not empty, calculate the amount of LP Tokens to be minted
    uint256 ethReservePriorToFunctionCall = ethReserveBalance - msg.value;
    uint256 minTokenAmountRequired = (msg.value * tokenReserveBalance) /
        ethReservePriorToFunctionCall;
```



```
require(
    amountOfToken >= minTokenAmountRequired,
    "Insufficient amount of tokens provided"
);
```

```
// Transfer the token from the user to the exchange
```



Search



Hi, MohammadLuqman!

```
lpTokensToMint =
    (totalSupply() * msg.value) /
    ethReservePriorToFunctionCall;

// Mint LP tokens to the user
_mint(msg.sender, lpTokensToMint);

return lpTokensToMint;
}
```

With adding liquidity now possible, let us also create an equivalent `removeLiquidity` function:

```
// removeLiquidity allows users to remove liquidity from the exchange
function removeLiquidity(
    uint256 amountOfLPTokens
) public returns (uint256, uint256) {
    // Check that the user wants to remove >0 LP tokens
    require(
        amountOfLPTokens > 0,
        "Amount of tokens to remove must be greater than 0"
    );

    uint256 ethReserveBalance = address(this).balance;
    uint256 lpTokenTotalSupply = totalSupply();
```



```
// Calculate the amount of ETH and tokens to return to the user
uint256 ethToReturn = (ethReserveBalance * amountOfLPTokens) /
    lpTokenTotalSupply;
uint256 tokenToReturn = (getReserve() * amountOfLPTokens) /
    lpTokenTotalSupply;

// Burn the LP tokens from the user, and transfer the ETH and tokens to the user
_burn(msg.sender, amountOfLPTokens);
payable(msg.sender).transfer(ethToReturn);
ERC20(tokenAddress).transfer(msg.sender, tokenToReturn);

return (ethToReturn, tokenToReturn);
}
```

Now, let's create a `pure` function that can perform the calculation of $x * y = (x + dx)(y - dy)$ to estimate how much ETH/Token would a user get back given they want to sell a certain amount of Token/ETH to the exchange.

```
// getOutputAmountFromSwap calculates the amount of output tokens to be received based on
// on  $xy = (x + dx)(y - dy)$ 
function getOutputAmountFromSwap(
    uint256 inputAmount,
    uint256 inputReserve,
    uint256 outputReserve
) public pure returns (uint256) {
    require(
        inputReserve > 0 && outputReserve > 0,
        "Reserves must be greater than 0"
    );

    uint256 inputAmountWithFee = inputAmount * 99;
```



```
uint256 numerator = inputAmountWithFee * outputReserve;  
uint256 denominator = (inputReserve * 100) + inputAmountWithFee;  
  
return numerator / denominator;  
}
```

With this in place, we can now create our two core swapping functions: `ethToTokenSwap` and `tokenToEthSwap`.

```
// ethToTokenSwap allows users to swap ETH for tokens  
function ethToTokenSwap(uint256 minTokensToReceive) public payable {  
    uint256 tokenReserveBalance = getReserve();  
    uint256 tokensToReceive = getOutputAmountFromSwap(  
        msg.value,  
        address(this).balance - msg.value,  
        tokenReserveBalance  
    );  
  
    require(  
        tokensToReceive >= minTokensToReceive,  
        "Tokens received are less than minimum tokens expected"  
    );  
  
    ERC20(tokenAddress).transfer(msg.sender, tokensToReceive);  
}  
  
// tokenToEthSwap allows users to swap tokens for ETH  
function tokenToEthSwap(  
    uint256 tokensToSwap,  
    uint256 minEthToReceive  
) public {  
    uint256 tokenReserveBalance = getReserve();  
    uint256 ethToReceive = getOutputAmountFromSwap(  
        tokensToSwap,
```



```
        tokenReserveBalance,  
        address(this).balance  
    );  
  
    require(  
        ethToReceive >= minEthToReceive,  
        "ETH received is less than minimum ETH expected"  
    );  
  
    ERC20(tokenAddress).transferFrom(  
        msg.sender,  
        address(this),  
        tokensToSwap  
    );  
  
    payable(msg.sender).transfer(ethToReceive);  
}
```

Deployment

With the code now done for both the contracts, let's deploy our contracts

Now we'll have to load our environment variables into the terminal's environment. For doing this, run the following command in your terminal

```
source .env
```



For deploying your token contract, make sure your terminal points to `foundry-app` and run this:

```
forge create --rpc-url $QUICKNODE_RPC_URL --private-key $PRIVATE_KEY --etherscan-api-key $ETHERSCAN_API_KEY --verify src/Token.sol:Token
```

Now copy the obtained address since we're gonna need it for our Exchange contract's constructor. Run the following command

```
forge create --rpc-url $QUICKNODE_RPC_URL --private-key $PRIVATE_KEY --constructor-arg <address of the token contract you just deployed> --etherscan-api-key $ETHERSCAN_API_KEY --verify src/Exchange.sol:Exchange
```



You might get messages saying that your contracts are already verified, even if they are not. This happens because you can't re-verify a contract identical to one that has already been verified. To circumvent this, you can run:

```
forge verify-contract <contract_address> <contract_name> --chain <chain_name>
```

Take a note of both contract addresses, as we will need them soon!

Test on Etherscan



Let's now test that everything works by playing around with our contracts on Etherscan.

Open up both contracts - the Token and the Exchange - on Sepolia Etherscan -

<https://sepolia.etherscan.io/>

You should have some `TOKEN` in the account you used to deploy the contract.

Adding Liquidity

Let's start off by trying to add some liquidity to the exchange. The Exchange relies on the ERC-20 Approve and Transfer Flow, so we need to go give it allowance for `TOKEN` from the Token contract.

On the `TOKEN` contract on Etherscan, go to the `Contract` tab and then `Write Contract`. Connect your wallet to that page, and click on `approve`

Input the Exchange contract address in the `spender` field, and set `amount` to `1000000000000000000`. Click on `Write` and confirm that transaction.

Once that transaction goes through, open up the `Exchange` contract on Etherscan - and go to `Contract` → `Write Contract` and connect your wallet there.

Click on `addLiquidity` and input some values for `payableAmount` and `amountOfToken`. I chose `0.1` and `10000000` respectively - but feel free to do whatever you'd like. Just make sure that `amountOfToken` is less than or equal to the amount you previously approved. Click on `View your transaction` and wait for the transaction to go through. You will notice that some `TOKEN`s were transferred out of your wallet into the Exchange, and then some new `ETH TOKEN LP` tokens were transferred into your wallet.



Swap

Now with some liquidity in our exchange, let's try to swap. I'm gonna first try to do an ETH → Token Swap. Click on the `ethToTokenSwap` function under `Write Contract` and input a `payableAmount` and `minTokensToReceive`. Since I only added `0.1 ETH` in liquidity, I put in `0.01 ETH` for the `payableAmount` and `0` for the `minTokensToReceive`.

Click `Write` and then `View your transaction`. Wait for it to go through. You will notice that you received some amount of `TOKEN` back to you in exchange for your ETH.

Similarly, let's try a `tokenToEthSwap`. I'm gonna do `10000` for `tokensToSwap` and then `0` for `minEthToReceive`. Click `Write` and then `View your transaction`.



NOTE: Normally, the minimum amount to receive is not set to zero. It is the job of the client (website, app, etc) to provide an estimate on how much tokens the user will receive before they even attempt a swap, and then the user agrees to a slippage percentage. So for example, if I was told to expect 100 TKN back for my swap, and I agree to 5% slippage - that means the minimum I will get is 95% of the original estimated amount i.e. 95 TKN - else the transaction will fail.

Same here - I got some ETH back for my tokens!



Removing Liquidity

Let's check how much LP tokens I have exactly and let's try to remove some of the liquidity.

Go to the `Read Contract` tab on the Exchange on Etherscan, and check the `balanceOf` for your own address. This should tell you the balance of how many LP tokens you own. For me, that was `1000000000000000000`.

Now, go back to `Write Contract` and then remove liquidity. Put down an `amountOfLPTokens` less than your balance - I did `500000000000` and then `Write` and `View your transaction`. Wait for it to go through.

Once the transaction is successful, you'll notice you get back some amount of ETH and some amount of `TOKEN` back.

Congratulations!

At this point, we've verified that everything in our exchange works perfectly! We've implemented the knowledge we gained from understand the equations behind AMMs - and we've successfully rebuilt Uniswap v1 from scratch.

If you have any doubts, or just want to celebrate your success, hit us up in the [Discord Server](#) - and we'll be there waiting for you!

[Submit Quiz](#)

Were you able to test out your exchange and make sure it works?

☐ Yes

☐ No

What do you think?

Post



WeightlessDovetail · 26 days ago

Yap



Reply



andidelouise · 5 months ago

Perfect



Reply



Babak Fa · last year

instructive



Reply



Bobi Dada · last year

Super



Reply



Soft-spokenGallop · 2 years ago

Perfect



Reply

Load More...

Learn	Opportunities	Community	Resources	Legal
Minis	Hackathons	Leaderboard	Testnet Faucet	Privacy Policy
Lessons	Open Source	Credentials	Level Up	Terms and Conditions
Courses & Degrees	Events	Discord		
		My Profile		

Subscribe to our newsletter

The latest news, articles, and resources, sent to your inbox weekly.

Subscribe



© 2025 LearnWeb3, Inc. All rights reserved.

